

Deep Learning Fundamentals

Hari 2 · Navasena Gen-ML Course · Batch 6

6 notebook · TensorFlow dan Keras · Google Colab

Navasena AI

Peta hari 2

Dasar belajar

nbo1 Neural network pertama (Fashion-MNIST)

Aktivasi

nbo2 Fungsi aktivasi dan optimizer

Gambar

nbo3 CNN dan transfer learning (CIFAR-10)

Urutan

nbo4 RNN dan LSTM

GPU

nbo5 Mixed precision, ONNX, TensorRT

Generatif

nbo6 Autoencoder, GAN, diffusion

Dua act pertama membahas cara satu neural network belajar. Sisanya memakai dasar itu untuk gambar, urutan, GPU, dan model generatif.

Dari hari 1 ke hari 2

ML klasik (hari 1)

- Feature dipilih dan dirancang manusia: kolom umur, gaji, durasi
- Data tabular, satu baris satu contoh
- Modelnya kecil, koefisiennya bisa dibaca satu per satu

Deep learning (hari 2)

- Feature dipelajari dari data mentah: pixel, urutan kata
- Gambar, teks, dan sinyal berurutan
- Modelnya berlapis, jumlah parameternya ratusan ribu ke atas

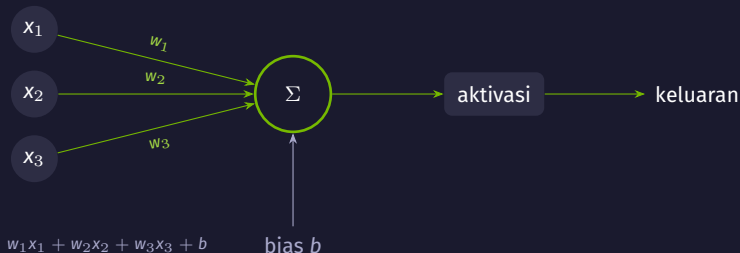
Yang tidak berubah

Alur kerjanya sama seperti hari 1: pisahkan data train, validation, dan test; bandingkan dengan baseline; pilih model di data validation. Yang berubah adalah bentuk modelnya dan dari mana featurenya datang.

Act 1

Bagaimana neural network belajar?

Dari satu angka loss ke pembaruan bobot



Intinya

Bobot adalah takaran: seberapa besar tiap masukan ikut dihitung. Bias menggeser hasilnya. Aktivasi menentukan bentuk keluaran neuron.

Dari satu neuron ke satu arsitektur



- Flatten menggelar gambar 28×28 menjadi satu baris berisi 784 angka.
- Dua hidden layer menyusun feature: layer berikutnya bekerja di atas keluaran layer sebelumnya.
- Output layer punya 10 neuron, satu untuk tiap kategori pakaian.

Yang berubah saat model belajar

Layer	Bobot	Bias	Parameter
Dense 128 (ReLU)	784×128	128	100.480
Dense 64 (ReLU)	128×64	64	8.256
Dense 10 (softmax)	64×10	10	650
Total			109.386

- Seratus ribu angka ini dimulai dari nilai acak, lalu diperbaiki selama training.
- Jumlah layer, jumlah neuron, dan learning rate kamu tentukan sebelum training; ketiganya bukan hasil training.

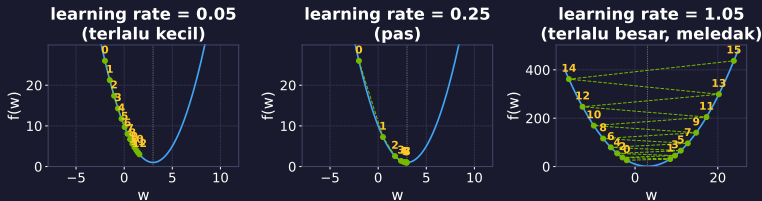
Loss: satu angka yang mengukur seberapa salah

$$\text{loss satu contoh} = -\log(p_{\text{kelas benar}})$$

- Model mengeluarkan satu probabilitas untuk tiap kelas. Loss hanya melihat probabilitas yang diberikan ke kelas yang benar.
- Probabilitas 0,85 untuk kelas yang benar memberi loss kecil; 0,05 memberi loss besar.
- Loss seluruh batch adalah rata-rata loss tiap contohnya. Angka inilah yang diperkecil saat training.

Namanya di Keras: `sparse_categorical_crossentropy`. Accuracy dilaporkan untuk kita baca; loss yang dipakai model untuk memperbaiki diri.

Gradient Descent pada $f(w) = (w-3)^2 + 1$



Intinya

Bentuk seluruh bukitnya tidak diketahui. Gradient menunjukkan arah perubahan loss di posisi sekarang; gradient descent memakai arah sebaliknya untuk memperbarui bobot, dan learning rate mengatur skala langkahnya.

Ketiga panel: fungsi sama, ukuran langkah berbeda.

Learning rate: ukuran langkah dan gejalanya

Terlalu kecil

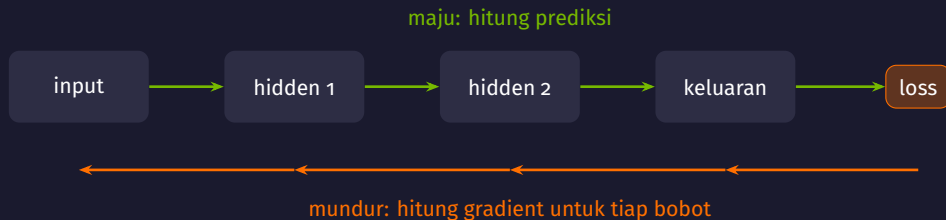
- Loss turun, tetapi sangat lambat
- Panel kiri: 12 langkah belum sampai ke dasar
- Epoch habis sebelum modelnya selesai belajar

Terlalu besar

- Langkahnya melewati dasar, lalu menjauh
- Panel kanan: nilai w makin jauh tiap langkah
- Loss naik-turun tanpa arah, atau menjadi NaN

Panel tengah, 0,25: dasarnya dicapai lewat beberapa langkah bertahap.

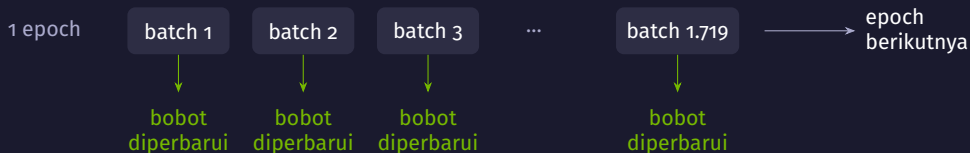
Jebakan: Adam memakai learning rate 0,001 sebagai nilai default; itu titik awal yang wajar, bukan angka yang cocok untuk semua data. Kalau loss tidak berubah atau menjadi NaN, periksa nilai loss, gradient, dan skala data lebih dulu. Menambah layer belum menjawab masalah itu.



Intinya

Setelah loss dihitung di keluaran, backprop menelusuri perhitungan kembali ke layer sebelumnya dan menghasilkan gradient loss terhadap tiap bobot. Optimizer memakai gradient itu untuk memperbarui bobotnya.

Epoch, batch, dan satu langkah perbaikan

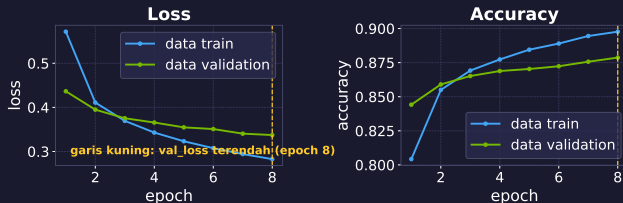


- Satu batch berisi 32 gambar yang dihitung bersama. Bobot diperbarui sekali untuk setiap batch.
- Satu epoch berarti seluruh data train dilewati sekali: 55.000 gambar dibagi 32 memberi 1.719 langkah perbaikan.
- Batch lebih besar membuat arah langkahnya lebih tenang, tetapi butuh memori lebih besar dan langkahnya lebih sedikit per epoch.

Angka ini dari nbo1: 55.000 data train, 5.000 validation, 10.000 test.

Kurva train dan validation dibaca berdampingan

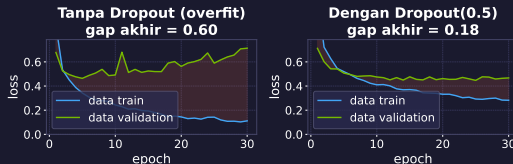
Kurva pelatihan: Dense(128) pada Fashion-MNIST (8 epoch)



- Kedua kurva loss masih turun bersama, dan akurasi validation berhenti di sekitar 0,88.
- Kalau loss validation mulai naik sementara loss train terus turun, model mulai menghafal data train.
- Pada run ini val_loss terendah jatuh di epoch terakhir, jadi 8 epoch belum cukup untuk melihat titikbaliknya.

Overfitting: melebarnya jarak train dan validation

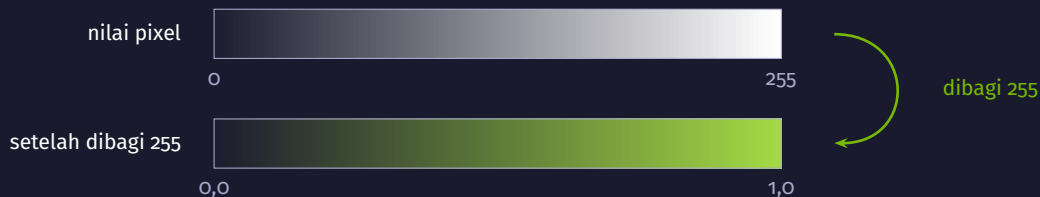
Overfitting vs Dropout(0.5) — 6000 sampel train, 30 epoch



- Model yang sama pada 6.000 sampel train. Selisih loss validation dan loss train di akhir: 0,60 tanpa dropout, 0,18 dengan Dropout (0.5).
- Saat training, dropout menolak sebagian aktivasi secara acak; early stopping berhenti sesuai batas patience. Periksa juga loss validationnya.

Jebakan: Loss validation bisa lebih rendah daripada loss train; salah satu penyebabnya dropout tidak aktif saat evaluasi.

Normalisasi input: dari 0–255 ke 0,0–1,0



- Masukan dengan rentang besar membuat gradientnya besar juga, sehingga langkah gradient descent sulit diatur dengan satu learning rate.
- Untuk pixel, pembagiya konstanta 255. Untuk data tabular, statistik scalingnya dihitung dari data train saja, lalu dipakai apa adanya ke validation dan test.

Evaluasi multi-kelas: akurasi saja belum cukup

Confusion matrix — test set (akurasi 87.4%)

Actual	T-shirt	849	1	8	45	5	2	84	6			
	Trouser	1	961		32	3		2	1			
	Pullover	24	1	701	16	171		86	1			
	Dress	18	2	7	915	29	1	23	5			
	Coat			40	36	861	1	61	1			
	Sandal						982	10	1	7		
	Shirt	152	1	59	49	85		647	7			
	Sneaker						44		946	10		
	Bag	10	1	6	8	9	6	4	5	951		
	Ankle boot							16	1	54	929	
		T-shirt	Trouser	Pullover	Dress	Coat	Sandal	Shirt	Sneaker	Bag	Ankle boot	
		Prediksi										

- Akurasi test 87,4% pada model Dense(128) di gambar ini: satu run, seed 42, data test disentuh sekali.
- Diagonal berisi prediksi yang benar. Sel di luar diagonal menunjukkan kelas apa tertukar dengan apa.
- Pasangan yang paling sering tertukar dihitung dari matriksnya, dan bisa berbeda tiap kali model dilatih ulang.
- Kelas dengan bentuk khas lebih jarang tertukar daripada kelas yang potongannya mirip.

Ringkasan Act 1

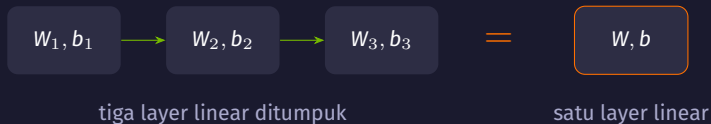
- Satu neuron menghitung jumlah berbobot dari masukannya, menambah bias, lalu melewatkannya ke aktivasi. Layer adalah kumpulan neuron seperti itu.
- Loss meringkas kesalahan menjadi satu angka. Backprop menghitung gradient loss terhadap tiap bobot, dan optimizer memakainya untuk memperbarui bobot.
- Learning rate menentukan ukuran langkah: terlalu kecil membuat training lambat, terlalu besar membuatnya meledak.
- Kurva train dan validation dibaca bersama. Jarak yang melebar adalah tanda overfitting, dan dropout serta early stopping menahannya.
- Untuk klasifikasi banyak kelas, confusion matrix menunjukkan kesalahan yang tidak terlihat dari angka akurasi.

Act 2

Kenapa butuh aktivasi?

Tanpa aktivasi, seratus layer sama saja dengan satu layer

Tanpa aktivasi, tumpukan layer tetap satu garis

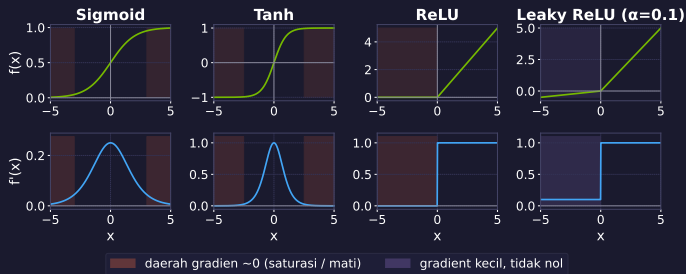


$$W_2 (W_1 x + b_1) + b_2 = (W_2 W_1) x + (W_2 b_1 + b_2)$$

- Hasil dua layer linear bisa ditulis ulang sebagai satu layer linear, jadi menambah layer tidak menambah bentuk yang bisa dipelajari.
- nbo2 memakai model tanpa aktivasi sebagai baseline linear, lalu membandingkannya dengan model yang beraktivasi.

Empat aktivasi dan turunannya

Fungsi Aktivasi dan Turunannya



- Baris atas fungsinya, baris bawah turunannya. Turunan inilah yang dipakai backprop.
- Daerah merah menandai tempat turunannya mendekati nol; bobot di sana hampir tidak berubah.



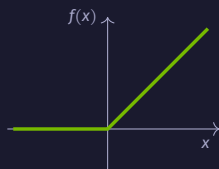
yang sampai ke layer 1: sekitar 0,016

Intinya

Turunan sigmoid paling besar 0,25. Contoh ini menganggap faktor dari bobot bernilai satu, jadi tiga layer sigmoid memberi sekitar 0,016. Pada jaringan sebenarnya faktor bobot ikut menentukan; ini salah satu penyebab gradient mengecil saat ditelusuri ke layer awal.

ReLU: turunannya tetap 1 di sisi positif

ReLU



$$f(x) = \max(0, x)$$

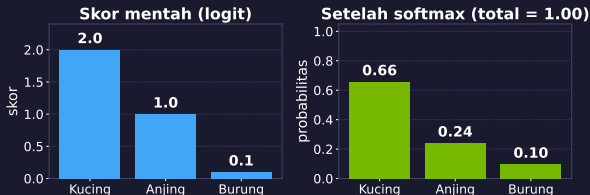
Leaky ReLU



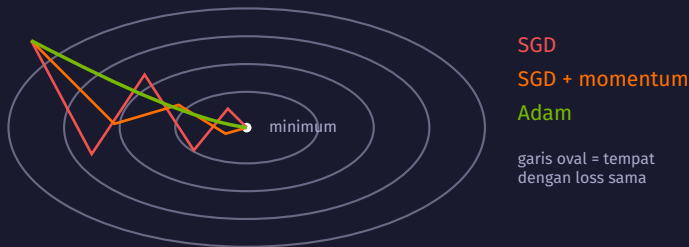
- ReLU meneruskan nilai positif apa adanya dan menolak yang negatif. Di sisi positif turunannya 1: pada jalur aktif, gradient tidak diperkecil oleh aktivasi ini.
- Perhitungannya juga murah: satu perbandingan dengan nol per neuron.
- Jika nilai sebelum ReLU selalu negatif pada data train, neuron terus menghasilkan nol dan tak menerima gradient lewat jalur itu: dead ReLU.
- Leaky ReLU memberi kemiringan kecil di sisi negatif, 0,1 pada gambar sebelumnya, sehingga masih ada gradient yang mengalir.

Softmax: skor mentah menjadi probabilitas

Softmax: dari skor mentah ke probabilitas



- Softmax bekerja pada seluruh output layer sekaligus: skor 2,0; 1,0; dan 0,1 menjadi 0,66; 0,24; dan 0,10, dengan total tepat 1.
- Selisih skor satu poin menjadi selisih probabilitas yang cukup besar, karena skornya lewat fungsi eksponensial dulu.
- Untuk dua kelas, satu keluaran dengan sigmoid sudah cukup. Untuk regresi, output layer dibiarkan tanpa aktivasi.



Intinya

SGD melangkah menurut kemiringan saat itu saja. Momentum menambahkan sisa arah langkah sebelumnya, sehingga jalurnya tidak banyak berbelok. Adam menyimpan rata-rata arah dan besar gradient, lalu menyesuaikan ukuran langkah tiap parameter.

Kapan mengganti aktivasi atau optimizer default

Cocok kalau

- banyak neuron ReLU berhenti aktif, keluarannya nol terus
- loss diam sejak epoch awal padahal datanya sudah dinormalisasi
- jaringannya dalam dan layer awalnya hampir tidak berubah

Kurang cocok kalau

- lossnya belum turun karena learning ratenya belum disesuaikan
- perbandingannya belum diuji di data validation
- modelnya masih underfit dan layernya memang terlalu sedikit

Jebakan: Dead ReLU biasanya berawal dari learning rate yang terlalu besar, bukan dari ReLU itu sendiri. Kalau lossnya tidak turun, ganti learning rate-nya dulu, sebelum arsitekturnya.

Ringkasan Act 2

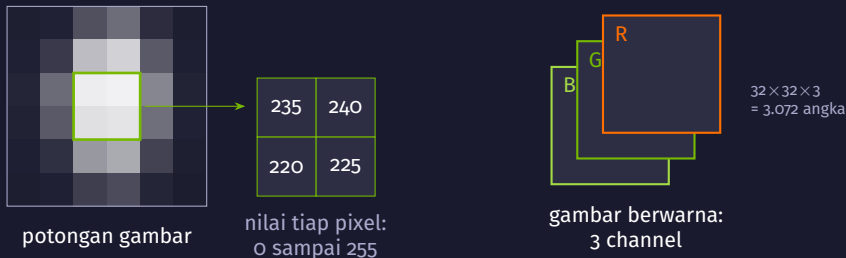
- Aktivasi adalah bagian yang membuat tumpukan layer bisa mempelajari pola melengkung. Tanpa aktivasi, hasil akhirnya tetap satu layer linear.
- Sigmoid dan tanh punya turunan yang mengecil di kedua ujung, sehingga gradient ikut mengecil saat ditelusuri ke belakang melewati banyak layer.
- ReLU menjaga turunannya tetap 1 di sisi positif dan murah dihitung, jadi dipakai sebagai pilihan awal untuk hidden layer. Leaky ReLU menyisakan gradient kecil di sisi negatif.
- Softmax dipakai di output layer klasifikasi banyak kelas; sigmoid untuk dua kelas; tanpa aktivasi untuk regresi.
- Optimizer hanya mengubah cara melangkah, bukan bukitnya. Adam menjadi titik awal yang wajar, dan learning ratenya tetap perlu diperiksa.

Act 3

Bagaimana komputer melihat gambar?

Satu filter kecil yang sama, digeser ke seluruh gambar

Gambar sampai ke model sebagai tabel angka



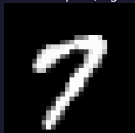
- Satu gambar Fashion-MNIST adalah 784 angka; satu gambar CIFAR-10 adalah 3.072 angka, karena tiap pixel punya nilai merah, hijau, dan biru.
- Dense layer membaca 3.072 angka itu sebagai satu daftar panjang: pixel bertetangga dan pixel di sudut berlawananan diperlakukan sama saja.

Konvolusi: satu filter kecil digeser ke seluruh gambar

Intuisi

Konvolusi manual: filter tepi vertikal vs horizontal

Gambar input (digit 7)



Filter tepi vertikal

-1	0	1
-1	0	1
-1	0	1

Hasil konvolusi



Filter tepi horizontal

-1	-1	-1
0	0	0
1	1	1

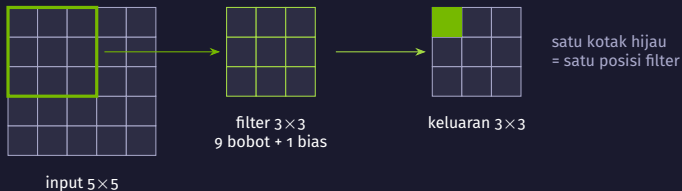


Intinya

Filter tepi vertikal bereaksi kuat hanya di tempat yang punya perubahan terang-gelap ke arah samping. Filter yang sama digeser ke semua posisi, jadi jawabannya berupa peta: di mana pola itu ditemukan.

Kedua filter ini ditulis tangan; di CNN, angka di dalam filternya dipelajari dari data.

Mekanisme konvolusi: satu jumlah berbobot per posisi

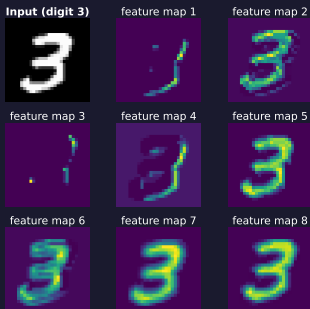


$$\text{keluaran}(r, c) = \sum_{i,j} w_{ij} x_{r+i, c+j} + b$$

- Bobot filter yang sama dipakai di setiap posisi. Dengan cara ini, model dapat mencari pola lokal yang sama di berbagai bagian gambar.
- `Conv2D(32, (3,3))` di `nbo3` memakai $32 \times (3 \times 3 \times 3 + 1) = 896$ parameter termasuk bias; satu Dense layer dari 3.072 nilai masukan ke 1.024 neuron memakai 3.146.752 parameter, juga termasuk bias.

Feature map: satu peta untuk tiap filter

Feature map layer conv1 (8 filter) — setelah 1 epoch



- Tiap filter menghasilkan satu feature map: kuning berarti filter itu bereaksi kuat di posisi tersebut.
- Delapan peta ini dari layer konvolusi pertama sebuah CNN kecil setelah satu epoch. Ada yang menyorot goresan tebal, ada yang menyorot latar, ada yang masih hampir kosong.
- Layer berikutnya bekerja di atas peta ini, bukan di atas pixel mentah, sehingga pola yang dikenalnya makin besar dan makin abstrak.

Pooling: menyusutkan peta, menyimpan yang paling kuat

1	3	2	4
5	6	1	0
2	1	7	3
0	4	2	8

feature map 4×4

ambil nilai
terbesar
→

6	4
4	8

keluaran 2×2

ukurannya seperempat,
tanpa satu pun
bobot yang dilatih

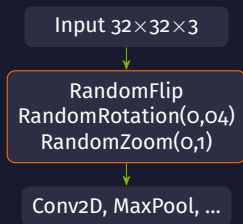
- `MaxPooling2D((2,2))` membagi peta menjadi petak 2×2 dan menyimpan nilai terbesar di tiap petak: di nbo3 ukurannya turun $32 \rightarrow 16 \rightarrow 8 \rightarrow 4$.
- Karena hanya nilai terkuat yang disimpan, pergeseran kecil pada gambar tidak banyak mengubah hasilnya, dan perhitungan layer selanjutnya jadi lebih ringan.

Arsitektur CNN nbo3 dari ujung ke ujung



- Bagian konvolusi menyusun feature dari pixel; bagian Dense di ujung memakai feature itu untuk memilih satu dari sepuluh kelas.
- Setiap blok melihat wilayah yang lebih luas dari gambar aslinya, jadi layer awal menangkap tepi dan layer akhir menangkap bagian objek.

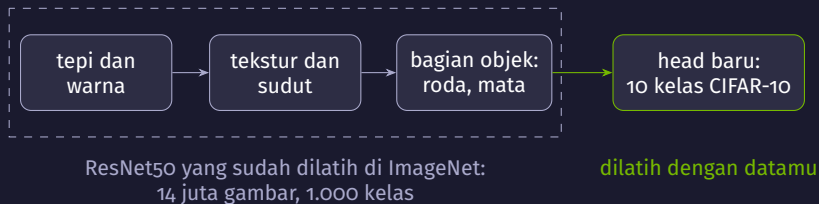
Augmentasi sebagai layer di dalam model



- Augmentasi menambah ragam data train tanpa gambar baru: gambar yang sama dibalik, diputar sedikit, atau di-zoom ulang tiap epoch.
- Ragamnya dipilih sesuai datanya: kucing yang dibalik tetap kucing, tetapi angka atau tulisan yang dibalik bisa berubah arti.

Aktif saat `fit()`, mati saat evaluasi.

Jebakan: Gambar train diacak sementara data validation tidak, jadi akurasi train bisa terlihat lebih rendah daripada akurasi validation di awal training. Itu efek augmentasi.



Intinya

Bagian yang mengubah gambar menjadi representasi disebut backbone; layer awalnya mempelajari hal yang umum: tepi, warna, tekstur. Backbone itu dipakai apa adanya, dan yang kita latih hanya classification head, bagian yang menghasilkan prediksi kelas.

Bobot backbone tetap, head dilatih



- `base_model.trainable = False` membekukan bobot backbone. Pada tahap ini, hanya bobot classification head yang diperbarui.
- Setiap gambar tetap diproses oleh seluruh backbone ResNet50. Yang tidak dilakukan pada tahap ini hanya pembaruan bobot backbone, jadi waktu per epochnya tetap perlu diukur.
- Pada alur latihan ini, fine-tuning dapat dilakukan setelah head dilatih: beberapa layer atas backbone dibuka, model dikompilasi ulang, lalu dilatih dengan learning rate yang lebih kecil.

Kapan transfer learning menolong, kapan tidak

Cocok kalau

- datanya sedikit: ribuan gambar, bukan jutaan
- resolusinya mendekati 224×224
- objeknya sejenis foto sehari-hari di ImageNet

Kurang cocok kalau

- gambarnya kecil, 32×32 seperti CIFAR-10
- domainnya jauh dari foto biasa: citra medis, radar
- datanya besar dan cukup untuk melatih CNN dari nol

Jebakan: Di nbo3, ResNet50 yang dibekukan tidak otomatis mengalahkan CNN kecil yang dilatih dari nol pada CIFAR-10: gambar 32×32 harus diperbesar dulu sebelum masuk backbone yang belajar di 224×224 . Bandingkan keduanya di data validation.

Ringkasan Act 3

- Gambar masuk ke model sebagai tabel angka. Konvolusi memanfaatkan kedekatan antar pixel yang diabaikan Dense layer.
- Satu filter kecil digeser ke seluruh gambar dan menghasilkan satu feature map. Bobot filter yang sama dipakai di setiap posisi, jadi parameternya jauh lebih sedikit.
- Pooling menyusutkan peta dengan menyimpan nilai terkuat, tanpa menambah parameter.
- Blok Conv → Pool ditumpuk sampai petanya kecil, lalu Flatten dan Dense mengambil keputusan kelas.
- Augmentasi sebagai layer hanya aktif saat training. Transfer learning memakai backbone terlatih dan melatih headnya, dan keunggulannya bergantung pada resolusi serta kedekatan domain.

Act 4

Bagaimana model membaca urutan?

Satu ringkasan pendek yang dibawa dari langkah ke langkah

Data berurutan: posisinya ikut membawa arti



- Mengubah urutannya mengubah artinya: urutan itu sendiri adalah informasi.
- Dense layer dan CNN memproses tiap contoh tanpa membawa apa pun dari contoh sebelumnya. Untuk urutan, model perlu tempat menyimpan yang sudah dibaca.
- nbo4 memakai gelombang sinus sebagai contoh kecil, lalu 50.000 review film IMDB yang dipotong menjadi 200 kata.

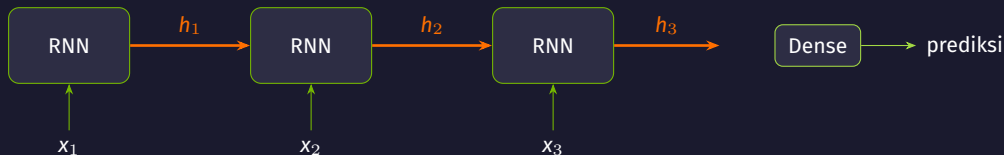
hidden state: ringkasan yang dibawa ke langkah berikutnya



Intinya

Modelnya membaca satu langkah pada satu waktu. Setiap langkah, ringkasan yang dibawa — hidden state — diperbarui dari hidden state sebelumnya dan masukan baru. Yang tersimpan bukan seluruh riwayat, hanya ringkasan sepanjang beberapa puluh angka.

Mekanisme RNN: bobot yang sama dipakai di tiap langkah



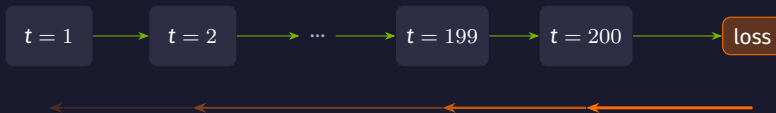
sel yang sama, bobot yang sama, dipakai ulang di setiap langkah

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

- W_x membaca masukan langkah ini, W_h membaca ringkasan langkah sebelumnya. Keduanya tidak berubah sepanjang urutan.
- Panjang hidden state ditentukan jumlah unit: SimpleRNN(32) di nbo4 membawa 32 angka antar langkah, dan keluaran langkah terakhir dipakai memprediksi.

Urutan panjang: pesan yang menipis di sepanjang rantai

Intuisi

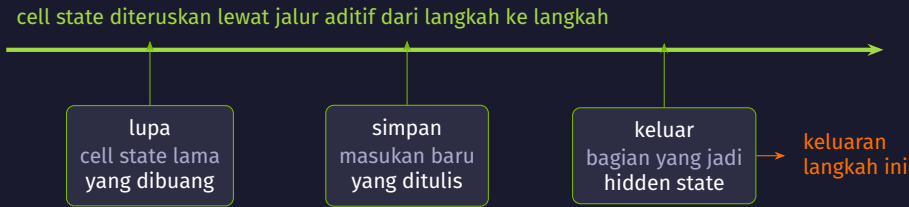


sinyal perbaikan dikalikan ulang di tiap langkah, dan makin tipis ke arah awal urutan

Intinya

Seperti permainan bisik berantai: makin panjang rantainya, makin sedikit pesan awal yang selamat. Review IMDB sepanjang 200 kata berarti 200 langkah, jadi kata di awal review hampir tidak lagi memengaruhi perbaikan bobot.

LSTM: tiga gerbang yang mengatur cell state



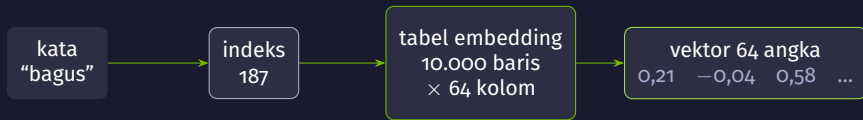
- Nilai gerbang dihitung dari input dan hidden state sebelumnya; bobot terlatih menentukan berapa banyak informasi dipertahankan, ditambahkan, atau diteruskan.
- Cell state diteruskan lewat jalur aditif; bagian yang dipertahankan diatur forget gate.
- Pada unit yang sama, LSTM punya lebih banyak parameter daripada SimpleRNN; waktu eksekusinya diukur sendiri.

GRU: versi ringkas dengan dua gerbang

Sel	Gerbang	Catatan terpisah	Parameter
SimpleRNN(32)	tidak ada	tidak	1.088
GRU(32)	update, reset	tidak	3.360
LSTM(32)	lupa, simpan, keluar	ya	4.352

- GRU menggabungkan gerbang lupa dan simpan menjadi satu gerbang update, dan tidak memisahkan cell state dari hidden state.
- Parameter di tabel dihitung untuk 32 unit dan satu feature masukan, seperti pada data sinus nbo4. Lebih sedikit bobot berarti lebih ringan per langkah.
- Pilihan antara GRU dan LSTM diselesaikan dengan mencoba keduanya dan membandingkan hasilnya di data validation, bukan dari jumlah gerbangnya.

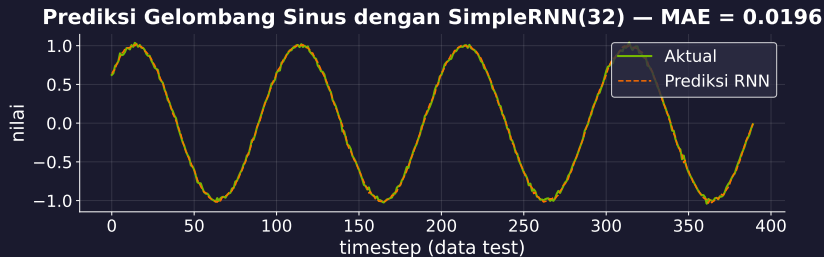
Embedding: dari indeks kata ke vektor



isi tabelnya dilatih bersama model, bukan ditetapkan sebelumnya

- Indeks kata hanyalah nomor urut di kamus. Kata bernomor 5.000 bukan dua kali lipat kata bernomor 2.500, jadi angkanya tidak boleh masuk langsung ke layer.
- Layer *Embedding* menukar setiap indeks dengan satu baris vektor, dan barisan vektor itulah yang dibaca LSTM langkah demi langkah.
- Di nbo4 semua review disamakan menjadi 200 langkah: yang pendek ditambah 0.

Hasil nyata: SimpleRNN memprediksi nilai berikutnya



- SimpleRNN(32) membaca 50 langkah terakhir dan memprediksi satu nilai berikutnya: MAE 0,0196 pada data test, satu run, seed 42, 20 epoch.
- Pembandingnya baseline “ulangi nilai terakhir yang dilihat”, dan di nbo4 baseline itu dihitung lebih dulu supaya angka modelnya punya arti.
- Pola berulang serapi ini memang mudah; deret nyata seperti penjualan atau sensor punya tren dan gangguan.

Kapan model urutan cocok, dan apa jebakannya

Cocok kalau

- urutannya bermakna, panjangnya puluhan sampai ratusan langkah
- datanya deret waktu dari sensor
- modelnya perlu kecil, misalnya untuk perangkat edge

Kurang cocok kalau

- konteks jauh menentukan; transformer di Modul 3 dan 4 lebih cocok
- urutannya bisa diringkas menjadi feature tabular
- barisnya sebenarnya tidak berurutan

Jebakan: LSTM tidak otomatis lebih baik daripada SimpleRNN. Pada urutan pendek seperti data sinus nbo4, keduanya bisa setara sementara LSTM memakai empat kali lebih banyak bobot. Yang menentukan adalah hasil di data validation.

Ringkasan Act 4

- Pada data berurutan, posisi ikut membawa arti, jadi modelnya perlu membawa sesuatu dari langkah sebelumnya.
- RNN menyimpan hidden state: satu ringkasan pendek yang diperbarui tiap langkah dengan bobot yang sama di sepanjang urutan.
- Makin panjang urutannya, makin tipis sinyal perbaikan yang sampai ke langkah awal. Itu vanishing gradient dalam bentuk waktu.
- LSTM menambahkan cell state dengan tiga gerbang terlatih; GRU melakukan hal serupa dengan dua gerbang dan bobot lebih sedikit.
- Teks masuk lewat layer Embedding yang menukar indeks kata menjadi vektor, dan panjang urutannya disamakan lebih dulu.

Act 5

Kapan GPU benar-benar diperlukan?

Dari perkalian matriks sampai TensorRT

Training berisi banyak perkalian matriks besar

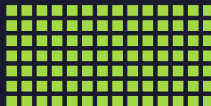


- Satu langkah training menghitung prediksi (maju), lalu menghitung gradient untuk tiap bobot (mundur). Keduanya berisi perkalian matriks seperti di atas.
- Jumlahnya banyak: benchmark nb05 melatih CNN pada 10.000 gambar CIFAR-10 dengan batch 64, selama beberapa epoch.
- Pekerjaan yang bisa dipecah seperti inilah yang bisa dikerjakan serentak.



CPU

sedikit unit hitung, tiap unit kuat dan serba bisa



GPU

banyak unit hitung sederhana yang bekerja serentak

Intinya

Analogi nb05: menghitung nilai ujian seribu siswa bisa dikerjakan satu guru yang cepat sekali, atau seribu kalkulator sederhana yang bekerja bersamaan. Pilihan kedua menang kalau pekerjaannya bisa dipecah.

Kapan GPU tidak memberi keuntungan

- **Modelnya kecil.** Untuk model kecil pada data tabular, overhead penggunaan GPU dapat lebih besar daripada penghematan waktu komputasinya.
- **Batchnya kecil.** Batch kecil dapat membuat kemampuan GPU kurang terpakai, terutama jika perhitungan per sampelnya ringan.
- **Datanya perlu dipindahkan.** Jika data masih berada di memori CPU, pemindahannya ke GPU ikut menambah waktu.
- **Pemanggilan pertama lebih mahal.** Pemanggilan pertama dapat mencakup inisialisasi runtime dan tracing graph, sehingga waktunya bisa lebih lama.

Jebakan: Warm-up mengurangi pengaruh inisialisasi pada pengukuran berikutnya. Di nb05, jalur CPU dan GPU sama-sama dijalankan sebelum waktu training dan inferensi diukur.

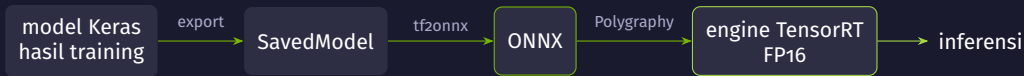
Mixed precision: hitung di float16, keluaran di float32

```
1 from tensorflow.keras import mixed_precision
2 mixed_precision.set_global_policy('mixed_float16')
3 Dense(10, activation='softmax', dtype='float32') # keluaran
```

- float16 memakai 16 bit per angka, float32 memakai 32 bit. GPU NVIDIA modern punya Tensor Core yang mengerjakan operasi float16.
- Bagian yang rawan dibiarkan di float32: keluaran softmax dan penjumlahan gradient. Karena itu output layernya diberi `dtype='float32'`.
- Percepatannya bergantung pada operasi dan hambatan komputasinya: keuntungannya terasa pada model dan batch yang cukup besar, dan besarnya diukur di nb05.

Di nb05 policynya dikembalikan ke `float32` setelah percobaan, supaya sel-sel berikutnya tidak ikut terpengaruh.

Dari training ke inferensi: ONNX lalu TensorRT

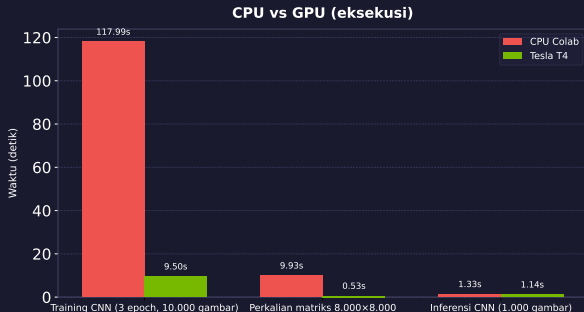


ONNX adalah format perantara: sekali model ada di sana, banyak mesin inferensi bisa membacanya tanpa peduli framework asalnya

```
1 !pip install "tensorrt>=10,<11" "tf2onnx>=1.17" onnx polygraphy
```

- TF-TRT, integrasi bawaan TensorFlow, dihapus sejak TensorFlow 2.18 (2.17 versi terakhir yang memuatnya) dan Colab sekarang memakai 2.18 ke atas.
- `tensorrt` dipin ke seri 10.x karena resep nb05 diverifikasi pada seri itu. `tf2onnx` minimal 1.17 supaya cocok dengan NumPy 2.

Benchmark pada konfigurasi kelas



nb05 di Colab, 7 September 2026: Tesla T4 lawan CPU runtime yang sama, mode penuh, setelah warm-up. Sumbu waktu skala log.

- **Training CNN** 3 epoch, 10.000 gambar: 118 s lawan 9,5 s.
- **Matriks** 8.000x8.000: 9,9 s lawan 0,5 s.
- **Inferensi** 1.000 gambar: 1,33 s lawan 1,14 s, hampir sama; batchnya kecil.
- **Mixed precision** di model ini: 14,0 s, lebih lambat dari float32 9,5 s.
- **TensorRT FP16** 14,5 ms lawan TensorFlow FP32 117,9 ms: dua konfigurasi berubah sekaligus.

Kapan GPU sepadan, dan apa jebakannya

Cocok kalau

- datanya gambar atau urutan panjang, modelnya berlapis
- batchnya cukup besar dan epochnya banyak
- banyak percobaan: ganti arsitektur, ulang training
- inferensinya melayani banyak permintaan sekaligus

Kurang cocok kalau

- data tabular kecil seperti hari 1
- satu prediksi sesekali di server biasa
- hitungannya ringan, waktunya habis untuk pindah data
- modelnya belum benar; masalahnya bukan kecepatan

Jebakan: GPU mempercepat hitungan, bukan memperbaiki model. Kalau loss tidak turun, yang perlu diperiksa adalah data, learning rate, dan arsitekturnya. GPU hanya membuat percobaan yang sama selesai lebih cepat.

Ringkasan Act 5

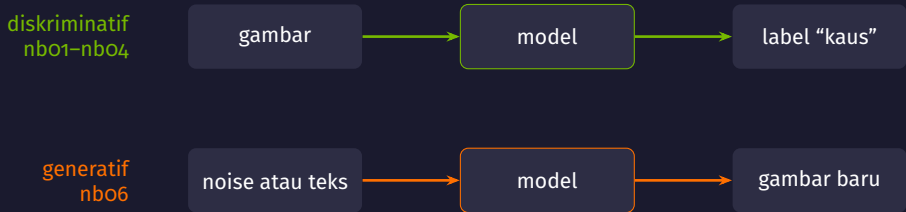
- Training deep learning berisi perkalian matriks besar yang bisa dipecah, dan itulah bentuk pekerjaan yang cocok untuk GPU.
- CPU punya sedikit unit hitung yang kuat; GPU punya banyak unit sederhana yang bekerja serentak.
- Keuntungannya hilang kalau modelnya kecil, batchnya kecil, atau waktunya habis untuk memindahkan data. Warm-up harus dikeluarkan dari pengukuran.
- Mixed precision menghitung di float16 dan menyimpan bagian yang rawan di float32; output layernya diberi `dtype='float32'`.
- Untuk inferensi, model diekspor lewat ONNX lalu dibangun menjadi engine TensorRT. Jalur bawaan TF-TRT sudah tidak ada di TensorFlow 2.18 ke atas.

Act 6

Bagaimana model menghasilkan data baru?

Memampatkan, berlomba, lalu membalik noise

Dua arah: memberi label, atau menghasilkan data



- Empat notebook pertama semuanya diskriminatif: masukan datang, model memberi label atau satu angka.
- Model generatif dilatih untuk hal lain: mengenali seperti apa data yang wajar, lalu mengambil contoh baru dari situ.
- Bahan dasarnya sesuatu yang sederhana, misalnya noise acak, yang diubah bertahap menjadi data yang rumit.

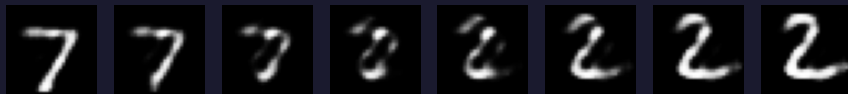


Intinya

Kalau gambar bisa dibangun ulang dari 32 angka, maka 32 angka itu memuat isi pentingnya. Decodernya kemudian bisa dipakai sendiri: masukkan 32 angka, keluar satu gambar. Di situlah autoencoder mulai menjadi generator.

Latent space: alamat 32 angka untuk tiap gambar

Interpolasi latent space: 7 → 2 (autoencoder, MNIST)



- Figur di atas: alamat satu digit 7 dan alamat satu digit 2 diambil, lalu jalan dari alamat pertama ke alamat kedua di-decode langkah demi langkah.
- Bentuk di antaranya tetap masuk akal. Berarti latent space menyimpan struktur, bukan hanya memampatkan gambar satu per satu.
- Titik acak yang jauh dari data justru memberi gambar kabur: autoencoder biasa tidak pernah diminta merapikan seluruh ruangnya. VAE menambahkan aturan itu.

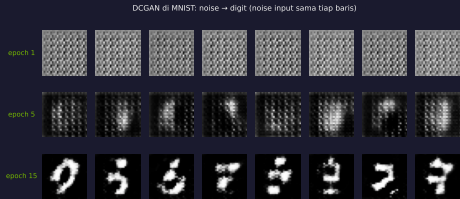
Autoencoder Dense pada MNIST, latent 32 angka, 10 epoch, seed 42.



Intinya

Generator membuat gambar dari noise. Discriminator belajar membedakan gambar dataset dan gambar buatan generator, dan penilaiannya menjadi bagian dari loss untuk melatih keduanya. Setelah training, gambar dibuat generator.

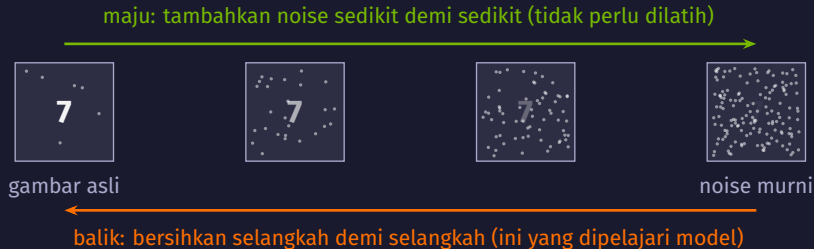
Digit yang lahir dari noise, epoch demi epoch



- DCGAN pada MNIST, 15 epoch, seed 42. Tiap kolom memakai noise masukan yang sama, jadi dari baris atas ke bawah yang berubah hanya generatornya: baris epoch 1 masih berbintik, baris epoch 15 sebagian sudah berbentuk digit yang bisa dikenali dan bukan salinan gambar dari dataset.

Jebakan: Mode collapse: generator bisa menemukan satu atau dua bentuk yang lolos terus, lalu berhenti belajar variasi lain. Gejalanya keluaran yang mirip satu sama lain, jadi yang diperiksa keragamannya.

Diffusion: noise ditambahkan bertahap, lalu dibalik



- Melatih model diffusion dari nol butuh data dan waktu jauh di luar satu sesi kelas.
- Inferensinya bertahap: satu gambar butuh beberapa sampai puluhan langkah pembersihan, bukan satu lompatan seperti GAN.
- Di nbo6 kita memakai model pretrained SD-Turbo dengan 4 langkah, dan menyimpan gambar di tiap langkah supaya prosesnya terlihat.

Peta keluarga generatif

Keluarga	Ide inti	Contoh
Autoencoder	Encoder membentuk representasi; decoder merekonstruksi input	Kompresi, denoising
VAE	Representasi probabilistik; sampel dari distribusi latent didekode	Generator wajah
GAN	Generator dan discriminator dilatih dengan tujuan berlawanan	StyleGAN
Diffusion	Belajar mengurangi noise pada berbagai tingkat; generasi bertahap	Stable Diffusion
Transformer autoregresif	Memprediksi token berikutnya dari token sebelumnya	GPT, Llama

Menuju Modul 4

Baris terakhir itu yang kita pakai di Modul 4. Pada model bahasa yang akan kita bahas, keluarannya berupa token teks.

Kapan model generatif cocok, dan apa jebakannya

Cocok kalau

- butuh contoh baru: menambah variasi data, mengisi bagian yang hilang
- butuh ringkasan padat: kompresi, denoising, deteksi anomali dari galat rekonstruksi
- hasilnya dinilai orang, bukan dicocokkan dengan satu jawaban

Kurang cocok kalau

- pertanyaannya klasifikasi atau prediksi satu angka
- jawabannya harus bisa diperiksa benar atau salah
- data trainnya sedikit dan sempit

Jebakan: Model generatif menghasilkan yang wajar menurut data trainnya, belum tentu yang benar. Akurasi tidak bisa dipakai menilainya, dan di Modul 5 kita lihat satu cara mengikat keluaran ke sumber yang bisa ditunjuk.

Ringkasan Act 6

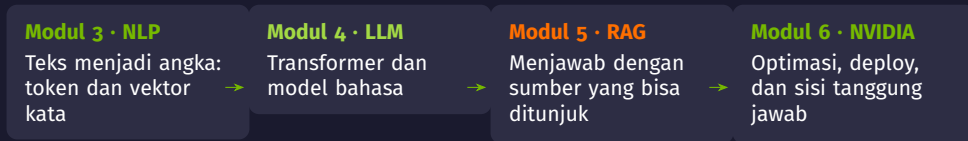
- Model diskriminatif memetakan data ke label; model generatif belajar seperti apa data yang wajar, lalu menghasilkan contoh baru.
- Autoencoder memampatkan gambar menjadi beberapa angka dan membangunnya ulang. Decodernya bisa dipakai sendiri sebagai generator.
- Latent space menyimpan struktur, terlihat dari interpolasi yang mulus. Titik acak di daerah kosong masih memberi hasil kabur.
- GAN melatih generator dan discriminator sebagai lawan tanding. Hasilnya bisa bagus, tetapi rewel dan rentan mode collapse.
- Diffusion membalik penambahan noise selangkah demi selangkah: mahal saat training, bertahap saat menghasilkan gambar.

Enam notebook hari ini

Notebook	Topik	Data
nbo1	Neural network pertama dengan Keras	Fashion-MNIST
nbo2	Fungsi aktivasi dan optimizer	Fashion-MNIST
nbo3	CNN dan transfer learning	CIFAR-10
nbo4	RNN, LSTM, dan embedding	Deret sinus dan review IMDB
nbo5	GPU, mixed precision, ONNX, TensorRT	CIFAR-10
nbo6	Autoencoder, GAN, dan diffusion	MNIST, SD-Turbo pretrained

Semuanya dijalankan di Google Colab dengan runtime T4, memakai `set_seed` supaya hasilnya bisa diulang, dan setiap model dibandingkan dengan baseline lebih dulu.

Apa yang menyusul sesudah hari 2



- Bekal hari 2 yang dipakai terus: loss, gradient descent, overfitting, embedding, dan cara memakai GPU.
- Model urutan hari ini adalah pintu masuk ke transformer, yang dipakai Modul 3 dan Modul 4 untuk teks.
- Jalur ONNX dan TensorRT hari ini muncul lagi di Modul 6, kali ini untuk model bahasa.

Latihan mandiri sebelum hari 3

- Tiap notebook punya blok **Latihan** di bagian akhir, dengan satu sel kosong untuk kodemu.
- Kerjakan minimal satu sebelum hari 3, dan pilih yang paling membuat penasaran.
- Contohnya: di nbo5 kecilkan jumlah gambar inferensi sampai keuntungan GPU hilang; di nbo6 ubah LATENT_DIM autoencoder menjadi 8 atau 64.
- Simpan hasil sebelum dan sesudah perubahan supaya perbandingannya jelas.

Kebiasaan yang dibawa dari hari 2

Sebelum menyimpulkan satu perubahan menolong, catat syaratnya: data mana, berapa epoch, seed berapa, dan baseline apa.

Terima kasih

Hari 2 selesai: dari satu neuron sampai model yang menghasilkan gambar

Navasena Gen-ML Course · Batch 6 · Deep Learning Fundamentals